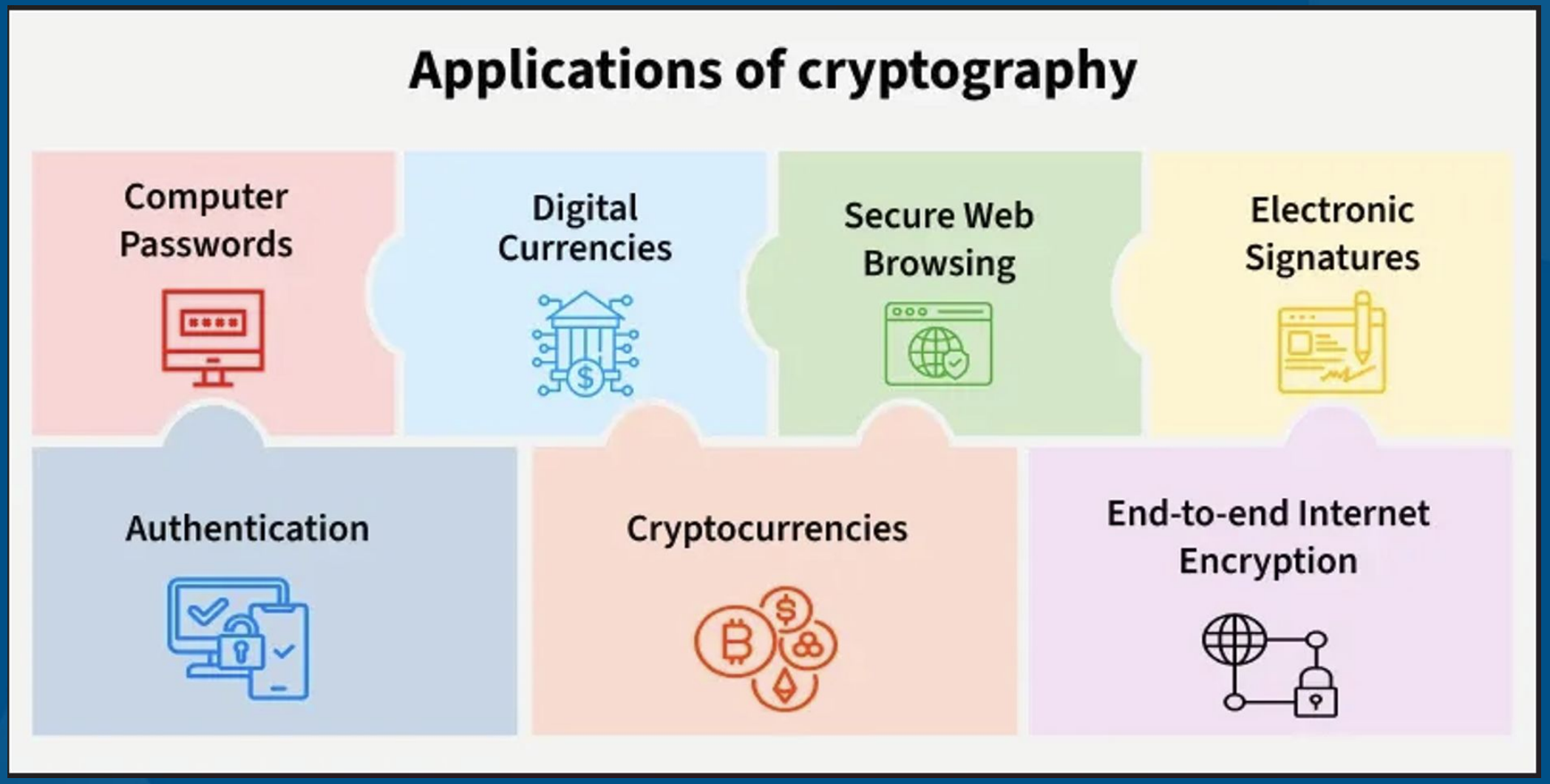
32-bit Modular Arithmetic and SIMD Vectorization

Authors: Duc Vinh Nguyen, Henry Zheng

Supervisors: Vincent Neiger, Hugo Passe

May 2026

Context


Applications of cryptography
Computer Passwords
Digital Currencies
Secure Web Browsing
Electronic Signatures
Authentication
Cryptocurrencies
End-to-end Internet Encryption

Project Objectives

Fundamental operation:

- **Integer multiplication modulo p:** compute (a . b) mod p with an integer a and an integer b in the set {0, … , p−1 }

$$a \ . \ b \bmod p$$

- **Scalar–vector multiplication modulo p:** compute the product of a fixed scalar b and a vector A, applying the modulo p operation to each component.

$$\left[a_1 \ . \ b \bmod p,\ a_2 \ . \ b \bmod p,\ \ldots,\ a_n \ . \ b \bmod p\right]$$

Project Objectives

Optimizing Modular Reductions

- **Instruction set architectures:** AVX2, AVX-512 and NEON
- **Objectives:** Improve FLINT's execution time for 32-bit arithmetic.
- **Impact:** Contribution to FLINT, crucial for cryptography and computer algebra.

Cost of modular reduction

Naive approach

On CPUs (Intel, AMD, ARM), the cost of a division is much higher than that of other arithmetic operations.

Division takes **~6 cycles per operation**, while multiplication and bit shifts take **~0.5–1 cycle per operation**.

$$a - \left\lfloor \frac{a}{p} \right\rfloor .p$$

Shoup's modular multiplication

Algorithm 1: Shoup's modular multiplication algorithm

Input : $A, B \in [0, p[$, we suppose that $p < \beta/2$
precompute $B' = \lfloor B\beta/p \rfloor$

Output: $R = AB \bmod p$

1 $Q \leftarrow \lfloor AB'/\beta \rfloor$;
2 $R \leftarrow (AB - Qp) \bmod \beta$;
3 **if** $R \geq p$ **then**
4 $\quad R \leftarrow R - p$;

Source: NTL

```
uint32_t q = ((uint64_t)a * b_precomp) >> 32;
uint32_t r = a * b - q * p;
if (r >= p)
    r -= p;
return r;
```

Mathematical theorem

Input Constraints Reminder

- **Word Size (β) :** Set to 2^{32} for 32-bit modular arithmetic
- **Input 1 (A) :** An integer defined over 32 bits
- **Input 2 (B) :** Integer within the range $[0, p[$
- **Modulus (p):** A prime number defined over 31 bits ($p < 2^{31}$)

Theorem

$$0 < AB - Qp < 2p$$

Vectorization Using AVX2 and AVX512

AVX2:

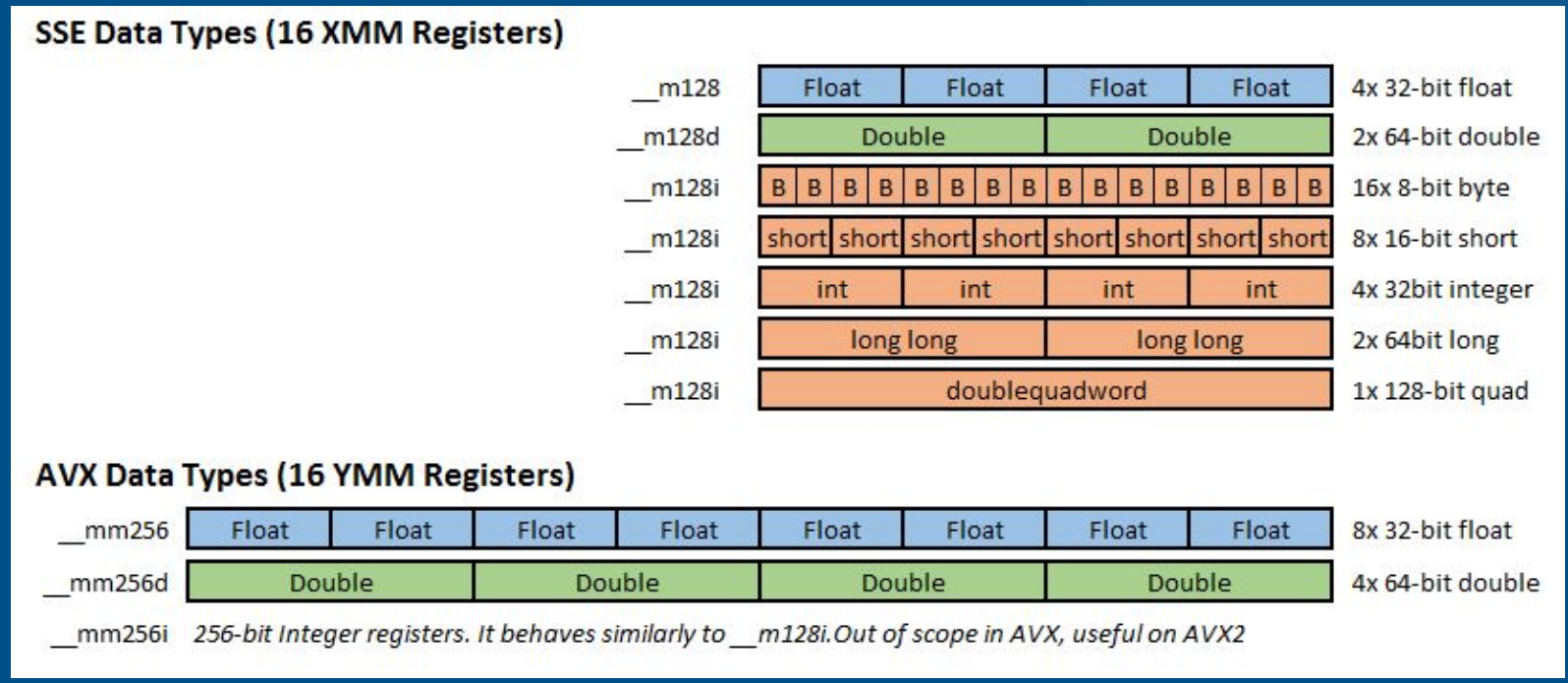
A **256-bit** register that can hold and process eight 32-bit integers at the exact same time.

(8x Speed Up)

AVX512:

A **512-bit** register that can hold and process sixteen 32-bit integers simultaneously.

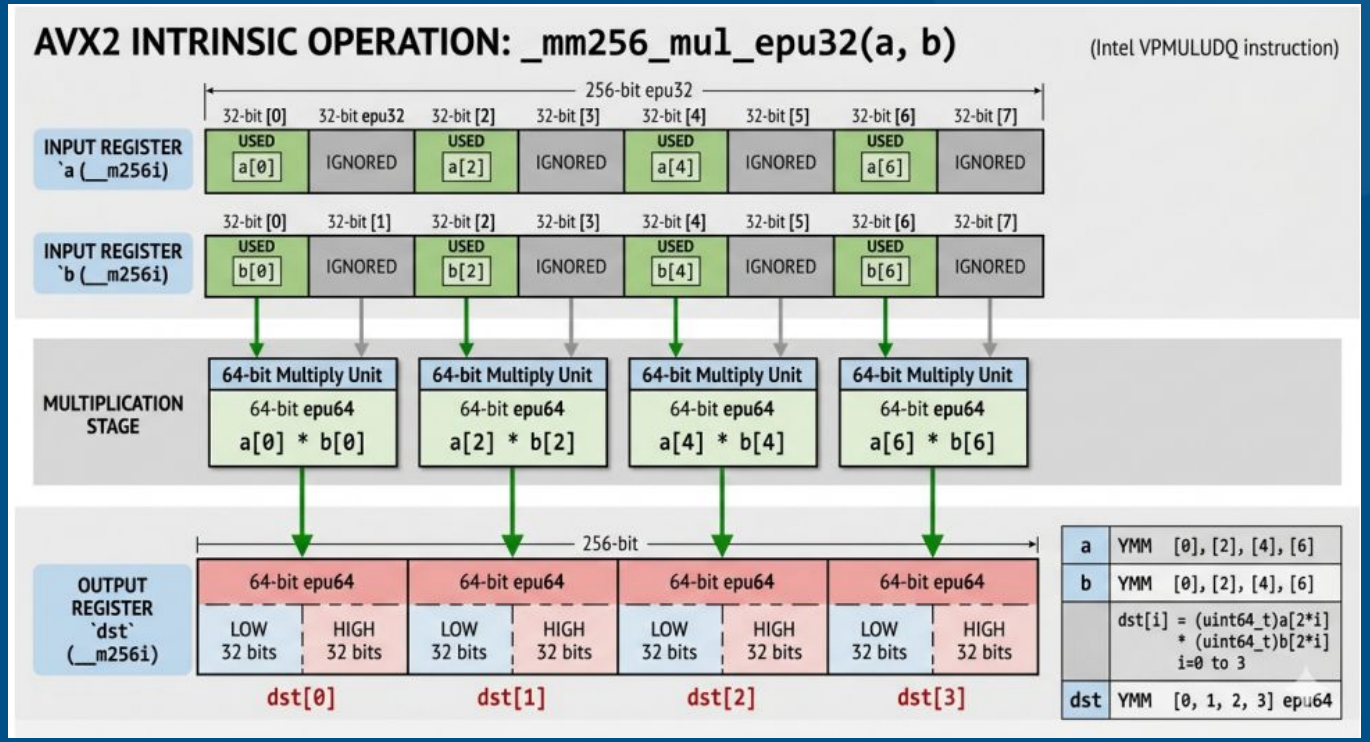
(16x Speed Up)



SIMD Implementation for AVX

Variant 1:

- **Variant 1:** uses mul_epu32 for every step (requires shuffling).

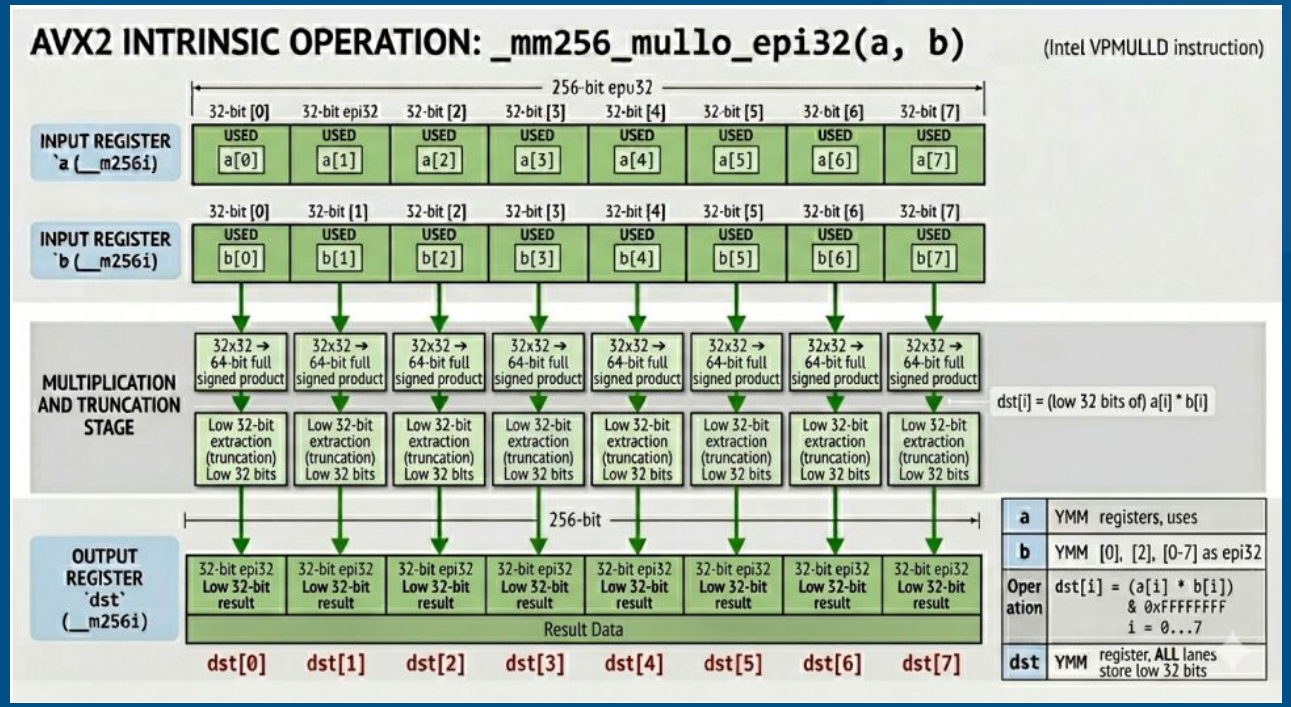


Source: Gemini

SIMD Implementation for AVX

Variant 2:

- **Variant 2:** mul_epu32 for the quotient and mullo_epi32 for the remainder



Source: Gemini

SIMD Implementation for AVX

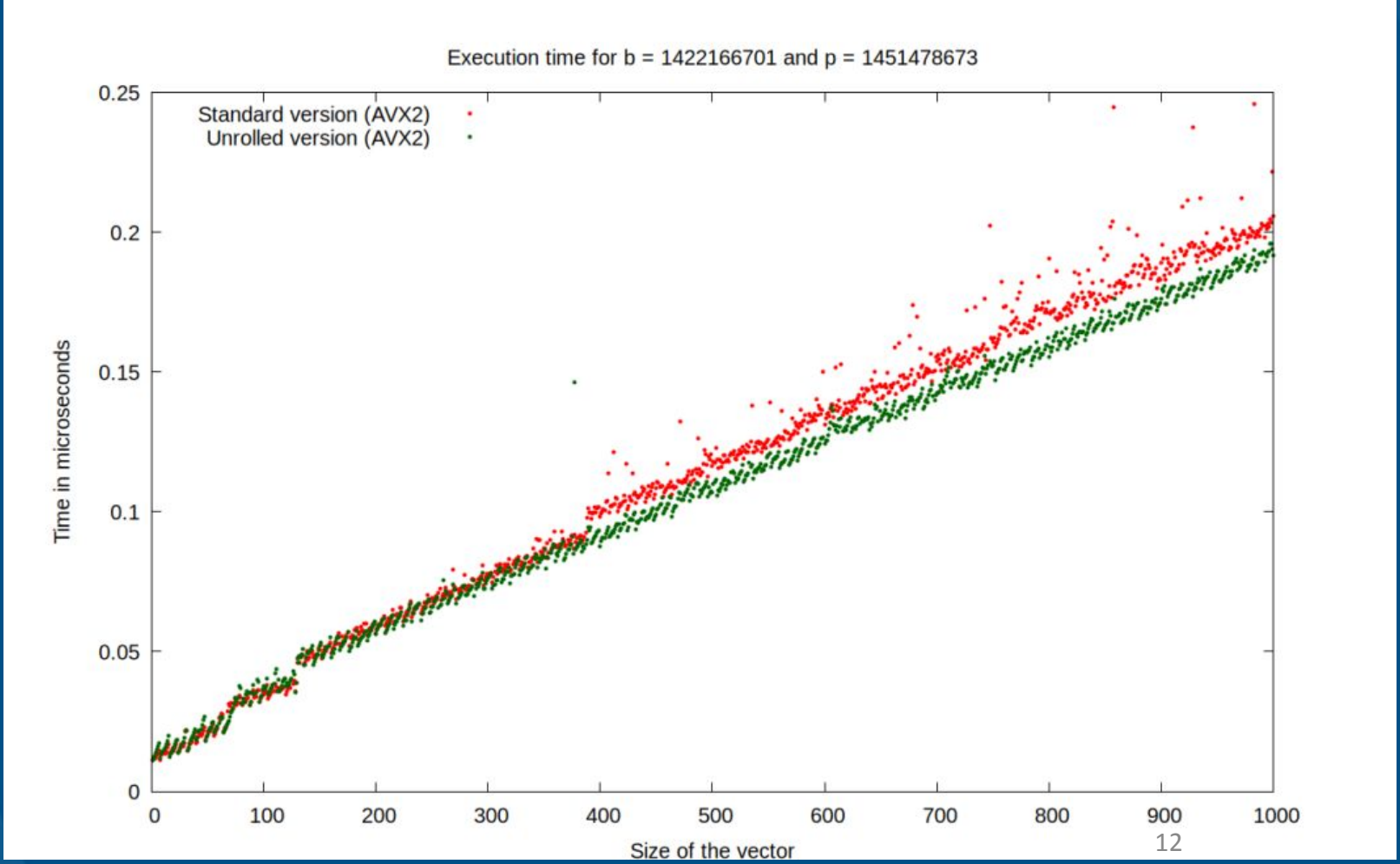
Variant 3:

- **Variant 3:** Emulation of a multiply-high function to group registers earlier in execution and mullo_epi32 for any stage.

```
__m256i vq = _mulhi_emul_avx2(va, vb_precomp);
__m256i vab = _mm256_mullo_epi32(va, vb);
__m256i vqp = _mm256_mullo_epi32(vq, vp);
__m256i vc = _mm256_sub_epi32(vab, vqp);
return _mm256_min_epu32(vc, _mm256_sub_epi32(vc, vp));
```

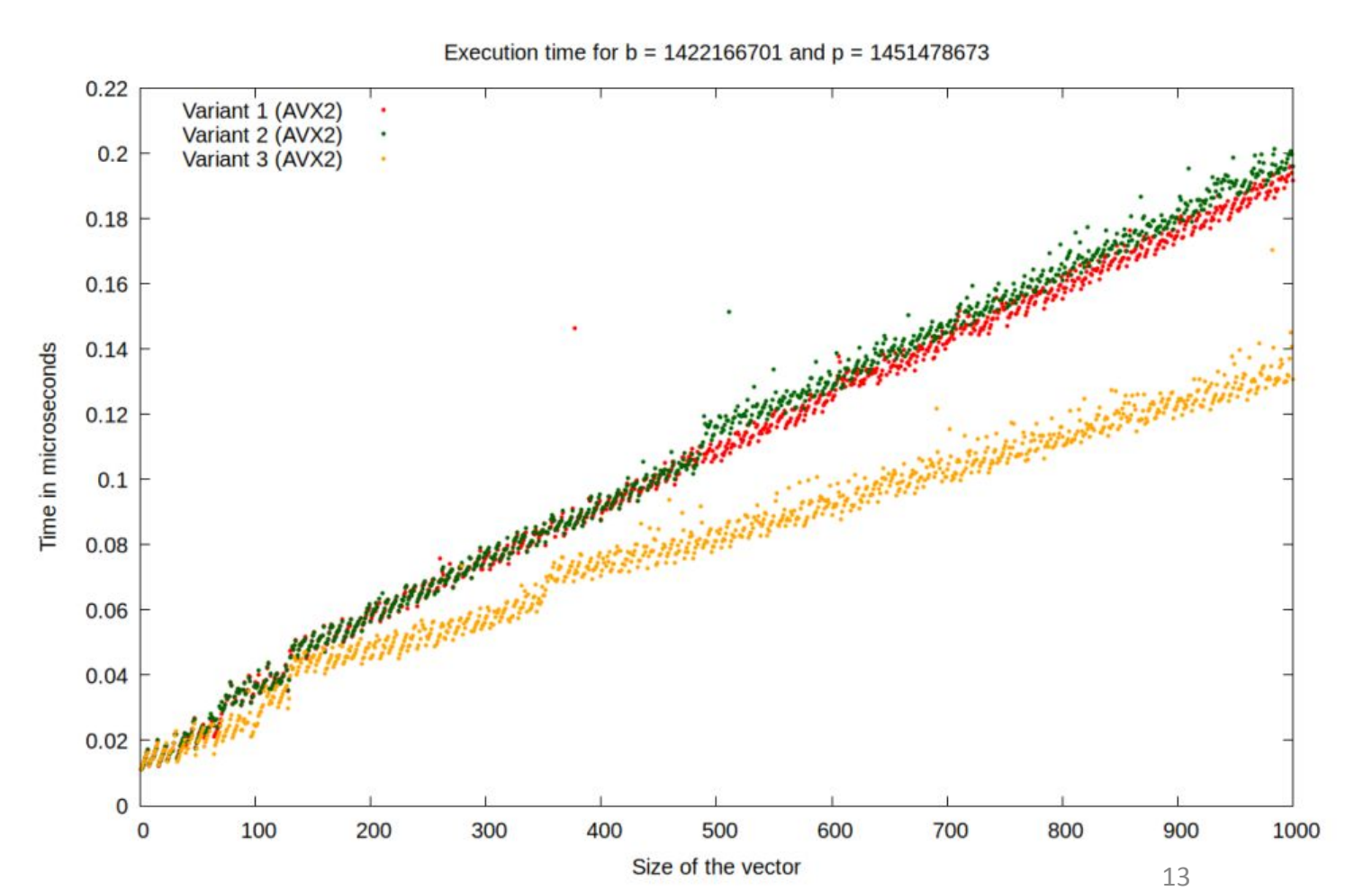
Performance: AMD Zen 4

Execution time (in microseconds) comparison: Standard vs. Unrolled (AVX2)



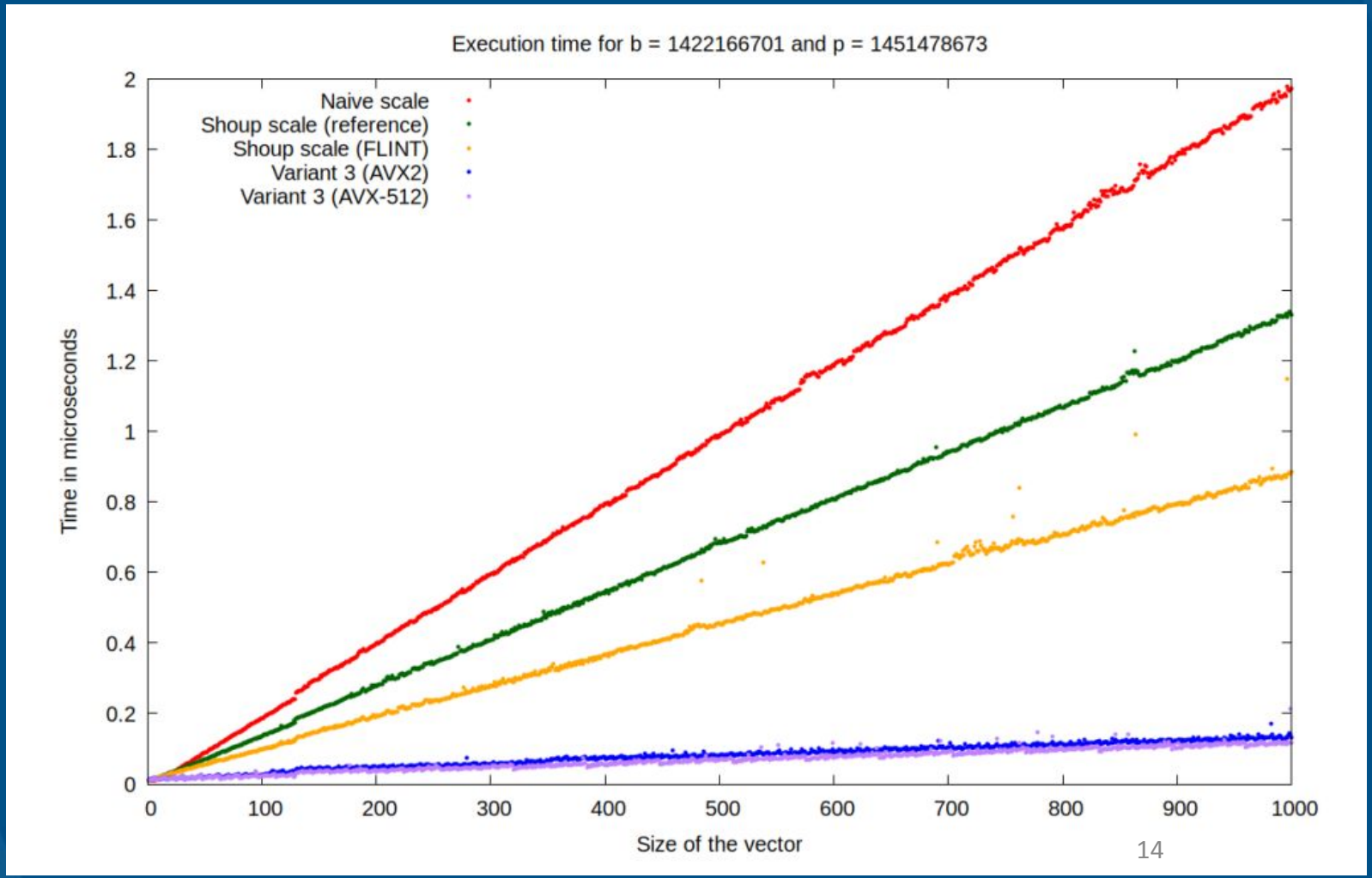
Performance: AMD Zen

Execution time (in microseconds) comparison of all variants (AVX2)



Performance: AMD Zen

Execution time comparison between different implementations (AVX)



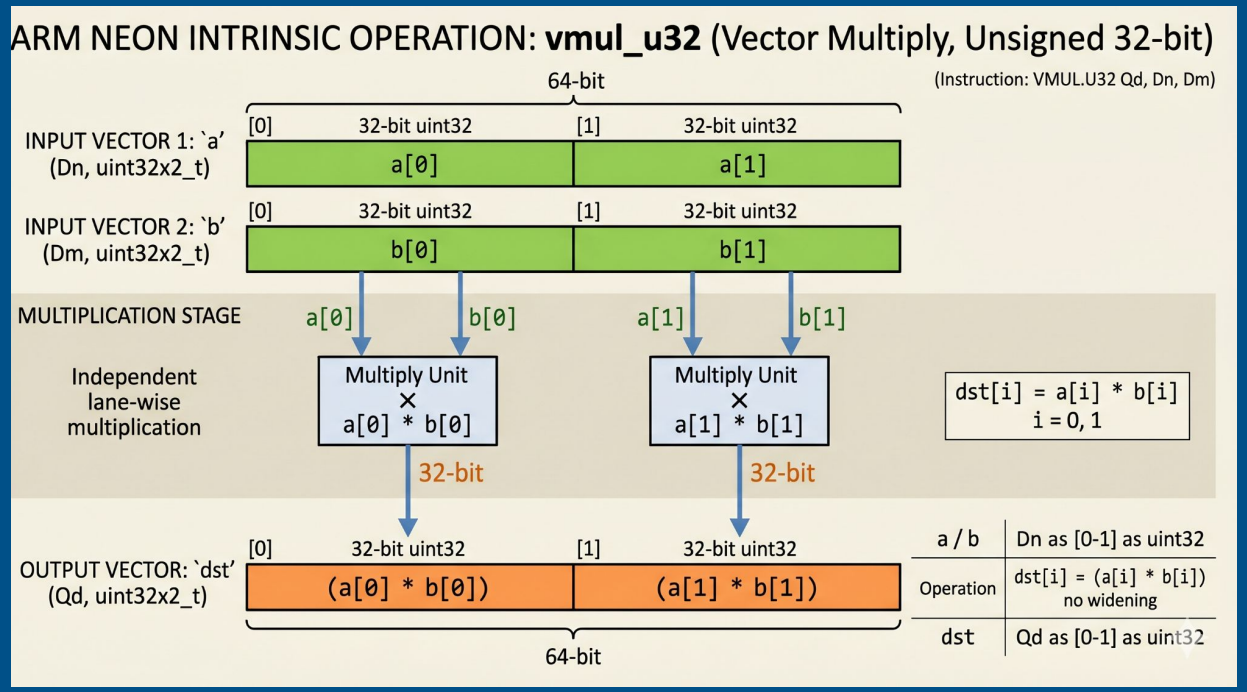
Conclusion for the AVX implementations

Algorithm	Average %	Acceleration Factor
FLINT	100.00%	1x
Shoup (AVX2)	16.86%	5.9x
Shoup (AVX-512)	14.35%	7.0x

Table 4: Average performance summary of FLINT in relation to Shoup AVX2 and Shoup AVX-512 implementations (AVX)

Shoup (AVX2) is roughly 5.9x faster than FLINT and Shoup (AVX-512) is 7.0x faster.

SIMD Strategy: ARM NEON (Apple M1)



128-bit Vector Processing

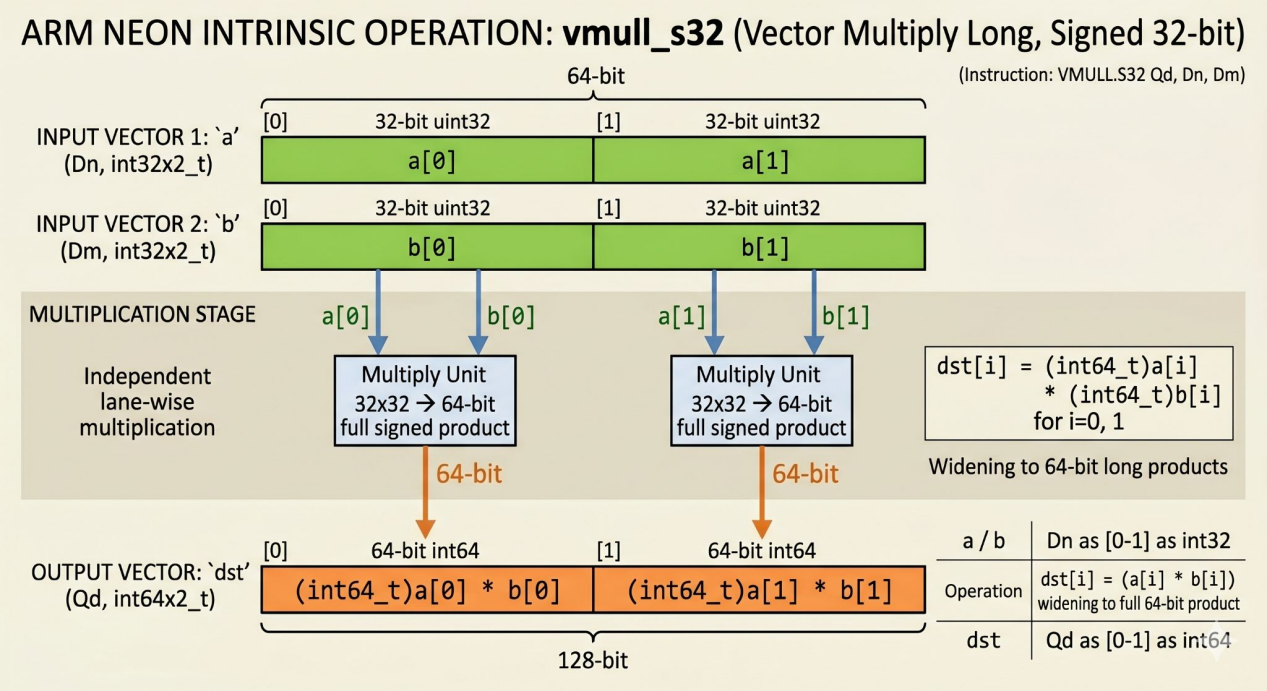
NEON registers hold two 32-bit integers.

Implementation uses:

- vmull_u32: 64-bit full products.
- vmul_u32: Lower 32-bit results.

Acceleration is theoretical 4x, halved to 2x due to alignment costs.

SIMD Strategy: ARM NEON (Apple M1)



128-bit Vector Processing

NEON registers hold two 32-bit integers.

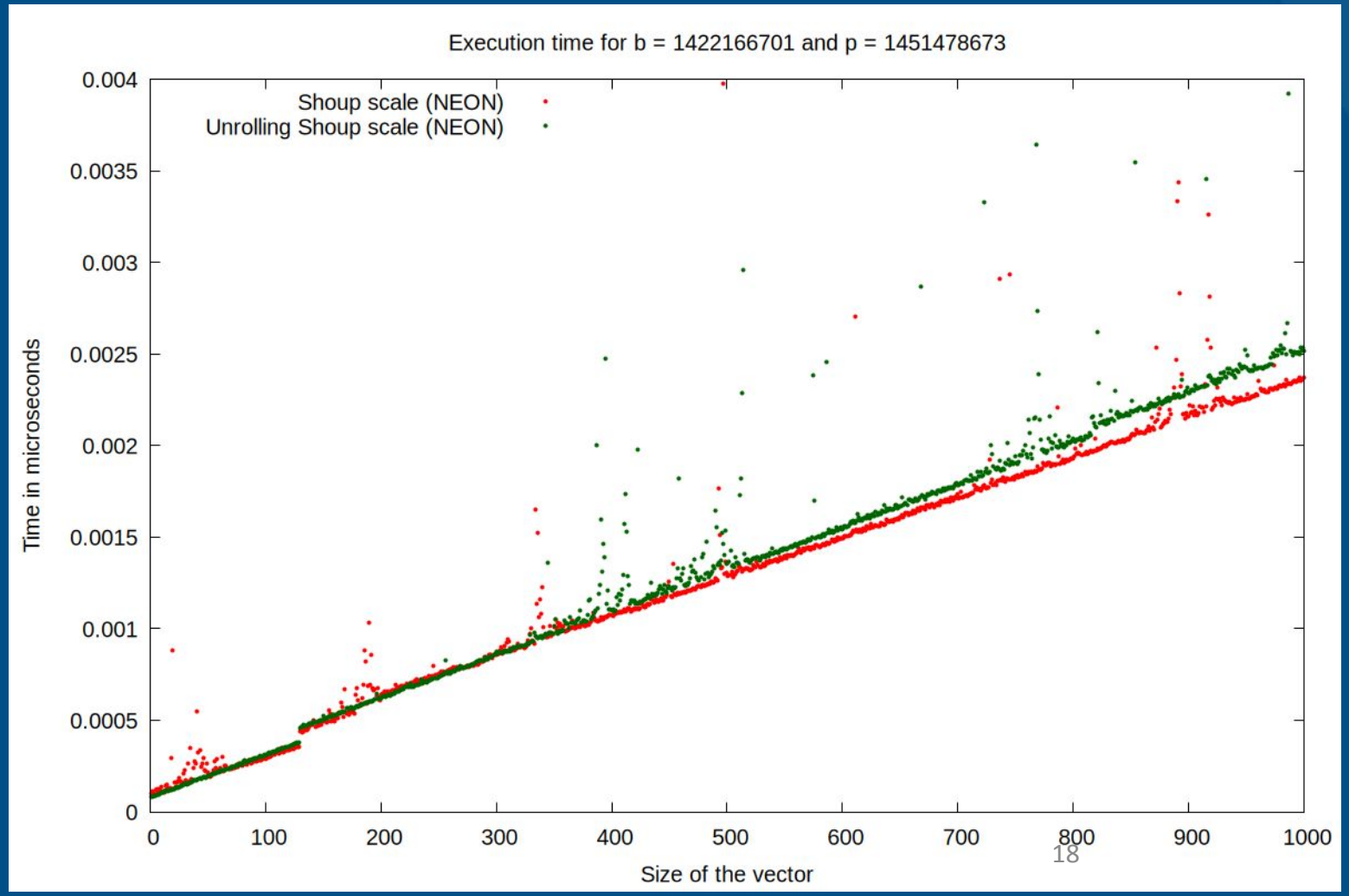
Implementation uses:

- vmull_u32: 64-bit full products.
- vmul_u32: Lower 32-bit results.

Acceleration is theoretical 4x, halved to 2x due to alignment costs.

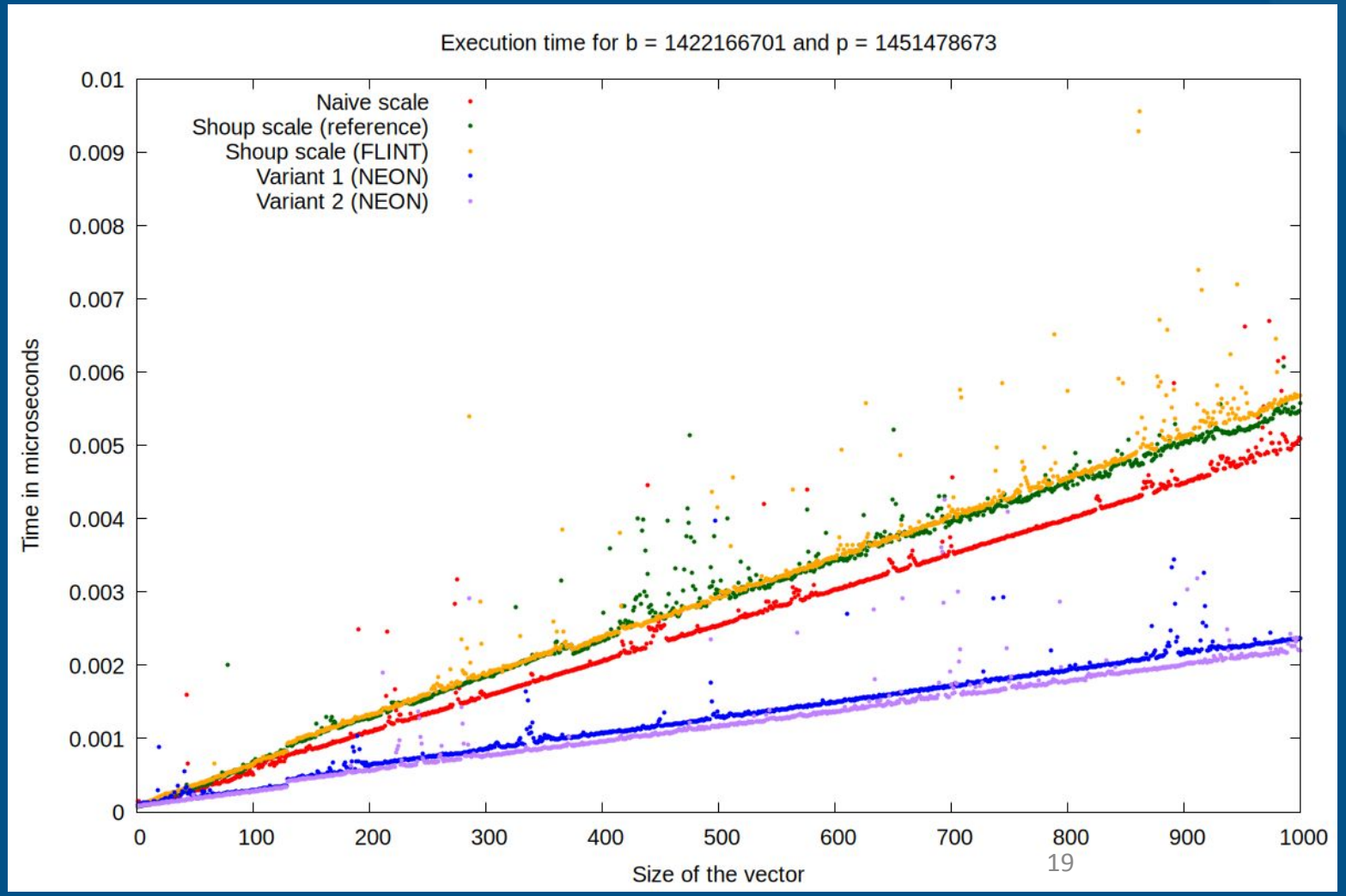
Performance: Apple M1

Execution time (in microseconds) comparison: Standard vs. Unrolled (NEON)



Performance: Apple M1

Execution time comparison between different implementations (NEON)



Analysis of results for the NEON implementations

Size	Naive	Shoup ref	FLINT	Variant 1 (NEON)	Variant 2 (NEON)
100	0.000514	0.000692	0.000653	0.000289	0.000279
200	0.001099	0.001274	0.001310	0.000637	0.000559
300	0.001573	0.001848	0.001871	0.000859	0.000763
400	0.002056	0.002389	0.002380	0.001077	0.000954
500	0.002542	0.003021	0.002929	0.001308	0.001162
600	0.003034	0.003422	0.003477	0.001492	0.001360
700	0.003630	0.003939	0.004094	0.001737	0.001733
800	0.003993	0.004420	0.005752	0.001927	0.001777
900	0.004488	0.005048	0.005128	0.002159	0.002000
1000	0.005093	0.005585	0.005683	0.002370	0.002208

- Results obtained are 100 times faster than those obtained with AVX.
- The Shoup scale reference function and the one implemented with FLINT are slower than the naive scale.
  $\longrightarrow$ Inconsistency regarding ARM processors
- The results obtained on ARM processors might be in milliseconds.
- The FLINT library may not be well-optimized for ARM processors.

Final Conclusion

- Successfully optimized and improved on FLINT's modular reduction implementation
- **For AVX :** Variant 3 outperformed FLINT's implementation by **6x to 7x,** depending on whether AVX2 or AVX-512 is used.
- **For NEON:** Variant 2 is the best performing implementation with an acceleration factor of **2.6x**